SIMATS ENGINEERING

CO2
ASSESSMENT TOOL 1

NAME:THUVIKSHA V

REG NO:1924260035

COURSE:ARTIFICIAL INTELLIGENCE

COURSE CODE:CSA1710

SUBMITTED TO: HERLIN.L.T

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently.

1. AI-Powered Medicine Delivery Drones

Scenario: A government uses AI-powered drones to deliver emergency medicines in a flood-affected region. Roads may become blocked, weather changes travel costs, and the drone has straight-line distance estimates and partial real-time updates.

i. Explain Greedy Best-First Search and its strengths and weaknesses under uncertainty.

Answer: Greedy Best-First Search chooses the next location with the smallest heuristic value h(n), such as straight-line distance to the destination. It can reach a promising route quickly and is often fast. However, it ignores the cost already travelled, may choose a route that looks close but is unsafe or expensive, and is not guaranteed to find the least-cost path. Inaccurate or outdated information can lead it into blocked or risky areas.

ii. Explain how A* handles the situation and balances cost and heuristic.

Answer: A* evaluates each location using f(n) = g(n) + h(n), where g(n) is the actual cost travelled so far and h(n) estimates the remaining cost. It considers both the route cost and the estimated distance to the destination. With non-negative costs and an admissible heuristic, A* can find an optimal route in a static map. Updated weather and path information must be incorporated into the costs and search.

iii. Analyze the impact of heuristic accuracy on both algorithms.

Answer: A more accurate heuristic guides both algorithms toward the destination. Greedy Best-First Search depends heavily on h(n), so a misleading estimate can cause a poor route choice. A* is also affected by heuristic quality, but g(n) helps it account for the cost already incurred. An admissible heuristic that does not overestimate the remaining cost supports A* optimality.

iv. Discuss how blocked paths affect search performance.

Answer: When a path becomes blocked, the drone must remove or mark that route as unavailable and search again. This can increase computation and delay delivery. Frequent updates may cause repeated replanning, so the system should use current sensor or communication data and prioritize safe alternative routes.

v. Recommend the most suitable algorithm for real-time constraints, optimality, and safety.

Answer: A* is the better default because it balances actual travel cost with estimated remaining cost and can find a least-cost route under suitable conditions. For real-time operation, it should be combined with rapid replanning, updated obstacle information, and safety constraints. If time is extremely limited, a bounded or incremental A* variant can help produce routes quickly.

2. Smart-City Traffic Signal Optimization

Scenario: A smart city must adjust signals across hundreds of intersections to reduce waiting time, fuel consumption, and congestion. Traffic changes dynamically, the search space is large, and local optima are possible.

Formulate the problem as an optimization problem and explain why traditional search is inefficient.

Answer: Represent a solution as signal timings and phase sequences for all intersections. Define an objective such as minimizing a weighted sum of total waiting time, fuel consumption, and congestion, subject to safety rules, minimum and maximum green times, and pedestrian requirements. Traditional exhaustive search is inefficient because the number of possible timing combinations grows rapidly and traffic conditions keep changing.

i. Explain Hill Climbing and identify its limitations.

Answer: Hill Climbing starts with a set of signal timings, evaluates nearby alternatives, and moves to a neighbor that improves the objective. It is simple and uses little memory, but it may stop at a local optimum, plateau, or poor solution. It can also struggle when a change at one intersection affects traffic at neighboring intersections.

ii. Explain local maxima, plateaus, and ridges in this scenario.

Answer: A local maximum (or local minimum for a cost function) occurs when nearby timing changes do not improve the current plan, even though a better plan exists elsewhere. A plateau occurs when many timing plans have similar performance, giving the search no clear direction. A ridge occurs when improvement requires coordinated changes across several signals, while changing only one signal appears unhelpful.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Answer: Simulated Annealing sometimes accepts a worse solution early in the search, helping it escape local optima; this willingness decreases as the temperature is lowered. A Genetic Algorithm maintains a population of timing plans and uses selection, crossover, and mutation to explore different combinations. Both can explore beyond immediate neighbors, although neither guarantees the global optimum within a fixed time.

iv. Recommend the best approach for scalability and adaptability.

Answer: A Genetic Algorithm is suitable for optimizing many interacting intersections because it explores multiple candidate plans at once. The objective can combine waiting time, fuel use, and congestion, while constraints enforce safe signal operation. To adapt to changing traffic, use recent traffic data, a limited computation time, and periodic re-optimization. Simulated Annealing is a simpler alternative when computation and implementation resources are limited.

3. Autonomous Mars Rover

Scenario: An autonomous rover explores unknown Martian terrain, collects samples, avoids hazards, has limited sensing, and updates its knowledge while moving.

i. Explain why an online search agent is needed instead of offline search.

Answer: An offline search agent normally plans using a known model of the environment. The rover does not have a complete map, so it cannot plan every step in advance. An online search agent acts, observes the result, updates its knowledge, and chooses the next action using newly discovered information.

ii. Analyze partial observability and dynamic-environment challenges.

Answer: Limited sensors may not reveal all rocks, craters, slopes, or surface conditions. Sensor noise and communication delays can make the rover uncertain about its surroundings. New obstacles or unexpected terrain may invalidate a planned route, so the rover must avoid unsafe actions and replan as information changes.

iii. Describe how the rover builds and updates its internal model.

Answer: The rover combines sensor readings, camera images, localization data, and observations from movement to build a local map. It records known safe areas, hazards, unexplored locations, and estimated travel costs. After each action, it compares expected and observed results and updates the map and its confidence in the information.

iv. Compare online search with uninformed and informed search strategies.

Answer: Uninformed search explores without using an estimate of how close a state is to the goal; examples include breadth-first search and depth-first search. Informed search uses a heuristic to guide exploration, as in Greedy Best-First Search or A*. Online search emphasizes acting and learning in an initially unknown environment. The rover can use heuristic guidance within an online strategy to reach sample sites more efficiently.

v. Justify a suitable approach considering uncertainty, adaptability, and safety.

Answer: An online, heuristic-guided approach with continuous sensing and replanning is most suitable. A* can plan through the currently known map, while the rover updates the map and replans when it discovers hazards. Safety rules should reject risky paths, and the rover should prefer routes with adequate clearance and reliable terrain estimates.

4. University Exam Timetabling as a CSP

Scenario: A university must schedule exams while avoiding student clashes, respecting room limits, prerequisite ordering, faculty availability, and special accommodations.

Formulate the problem as a Constraint Satisfaction Problem (CSP).

Answer: A CSP consists of variables, domains, and constraints. Each exam is a variable. Its domain contains possible time slots and rooms. Constraints rule out assignments that cause a student to have two exams at once, exceed room capacity, violate required subject order, conflict with faculty availability, or fail to provide approved accommodations.

i. Define variables, domains, and constraints.

Answer: Variables: the time slot and room assigned to each exam. Domains: available time slots and suitable rooms for each exam. Hard constraints: no student overlap, sufficient room capacity, room availability, faculty availability, required subject ordering, and accommodation needs. Soft preferences, such as avoiding several exams on one day, may be used to rank valid timetables.

ii. Explain backtracking search.

Answer: Backtracking assigns a time and room to one exam at a time. After each assignment, it checks whether the constraints remain satisfied. If a conflict occurs, it undoes the latest assignment and tries another value. The process continues until a complete valid timetable is found or all possibilities have been tried.

iii. Explain how constraint propagation, such as forward checking, improves efficiency.

Answer: After assigning an exam, forward checking removes incompatible time slots or rooms from the domains of related unassigned exams. If any variable has no valid values left, the algorithm backtracks immediately instead of exploring a doomed partial timetable. This reduces unnecessary search.

iv. Analyze real-world challenges and recommend a scalable strategy.

Answer: Challenges include many courses and students, limited rooms, changing faculty availability, special accommodations, and conflicting preferences. A scalable strategy combines backtracking with constraint propagation, the minimum-remaining-values heuristic to choose the most restricted exam first, and degree-based tie-breaking. Optimization methods can then improve soft preferences without violating hard constraints.

5. Strategic Game AI: Minimax and Alpha-Beta Pruning

Scenario: A gaming company needs an AI opponent that evaluates many possible game states and makes strong decisions within strict time limits.

i. Explain how Minimax models the problem.

Answer: Minimax represents the game as a tree of possible moves. The AI is the maximizing player and tries to choose moves with the highest utility, while the opponent is assumed to choose moves that minimize that utility. Values are backed up from terminal or depth-limited states to select the best available move.

ii. Analyze computational challenges of large game trees.

Answer: If each state has b possible moves and the search depth is d, ordinary Minimax may examine about O(b^d) states. The tree can therefore grow exponentially, requiring substantial time and memory. A strict real-time limit may prevent the AI from searching all the way to terminal states.

iii. Explain how Alpha-Beta pruning improves efficiency.

Answer: Alpha-Beta pruning avoids exploring branches that cannot change the final Minimax decision. Alpha is the best value currently guaranteed to the maximizing player; beta is the best value guaranteed to the minimizing player. When alpha is greater than or equal to beta, the remaining branch can be pruned. With good move ordering, pruning can greatly reduce the number of evaluated states while preserving the same decision as Minimax at the searched depth.

iv. Design an evaluation function and justify its factors.

Answer: A simple evaluation function is: Score = w1(Material advantage) + w2(Objective control) + w3(Strategic position) + w4(Resource advantage) + w5(Threat advantage) − w6(Immediate danger). The weights reflect the game's priorities. Material and resources measure available strength, objective control measures progress, position rewards useful locations, and threat and danger terms account for tactical risks.

v. Recommend a strategy balancing optimality and real-time performance.

Answer: Use depth-limited Minimax with Alpha-Beta pruning, strong move ordering, and an evaluation function. Iterative deepening searches depth 1, then 2, and so on, allowing the AI to return its best move if time expires. A transposition table can reuse previously evaluated states. This approach preserves Minimax's result at the completed depth while making better use of the time limit.

Conclusion

Search and optimization methods must be selected according to the environment, cost of decisions, uncertainty, and available computation. A* is useful for cost-aware routing, metaheuristics for large optimization spaces, online search for unknown environments, CSP methods for timetabling, and Minimax with Alpha-Beta pruning for strategic games.